

Text + Voice HCI Assessment

(CLO-3)

BSSE23B

Dr. Muhammad Asif

Human Computer Interaction

SE305T (Spring-26)

Individual proctored Assessment

LT2-BlockB

Information Technology University (ITU) – Lahore

20-May-2026

Time= 4:00pm - 5:30pm

Total Marks=100

The Scenario

A university has commissioned you to build NEXUS — an AI-powered student wellbeing advisor. NEXUS must accept voice and text input, track how students are feeling across a session using a 6-tier wellbeing scale, identify what kind of support they need (academic, personal, administrative, etc.), flag at-risk students automatically, and generate a session intelligence report for the counselling team.

Assessment Rules & Constraints

Permitted

- Python standard library + listed packages
- Your own handwritten or printed notes
- Official library docs (offline only)
- The starter scaffolding printed on this paper
- Running and testing code locally

Strictly Forbidden

- AI tools — ChatGPT, Copilot, Gemini, etc.
- Looking at another student's screen or code
- Any paper from a previous class session
- Internet searches of any kind
- Reusing code not written by you today

Approved Packages — install before the session

```
openai-whisper    sounddevice    scipy    numpy    nltk    textblob
matplotlib        anthropic
```

Stage 1 — Input Layer: Voice + Text Capture & Transcription

20 minutes | 20 marks total

Build the input foundation of NEXUS. Every function in this stage must use the exact names specified — your submission will be auto-tested against these signatures.

Mandatory Function Names for This Stage

`nexus_capture_audio()` `nexus_transcribe()` `nexus_get_input()` — any other names score zero.

Q1.1 — Audio Capture

Write `nexus_capture_audio(seconds=7, sample_rate=16000)` that records from the microphone and returns a `numpy float32` array. Include a print statement telling the student how many seconds remain. Add a docstring explaining what the function does. **[8 marks]**

```
import sounddevice as sd
import numpy as np

def nexus_capture_audio(seconds=7, sample_rate=16000):
    """
    # YOUR DOCSTRING HERE — describe what this function does
    """
    # YOUR CODE HERE
    pass
```

Q1.2 — Whisper Transcription

Write `nexus_transcribe(audio_array, sample_rate)` using Whisper model 'small' (not 'base' — NEXUS needs better accuracy). Return a dict: `{'text': str, 'language': str, 'confidence': str}`. Set confidence to 'high' if text length > 10 chars, else 'low'. **[7 marks]**

Q1.3 — Unified Input with Source Tracking

Write `nexus_get_input(turn_number)` that shows the turn number in the prompt, accepts text or voice, and returns a dict: `{'text': str, 'source': str, 'turn': int, 'word_count': int}`. Include word count of the input. This is different from the customer service version — students are more verbose. [5 marks]

☑ Stage 1 Verification — run this before proceeding:

```
result = nexus_get_input(turn_number=1)
assert 'text' in result and 'source' in result
assert 'turn' in result and 'word_count' in result
assert result['source'] in ('voice', 'text')
print('Stage 1 OK:', result)
```

Stage 2 — Wellbeing Engine: 6-Tier Emotional State Tracker

20 minutes | 25 marks total

NEXUS supports university students — a uniquely varied emotional audience. A standard 3-level or 5-level scale is insufficient. You will implement a 6-tier wellbeing tracker with an urgency flag system that identifies students who need immediate human intervention.

NEXUS 6-Tier Wellbeing Scale

 THRIVING ≥ 0.6	 CONTENT 0.2-0.59	 NEUTRAL -0.19-0.19	 STRESSED -0.4--0.2	 DIS-TRESSED -0.6--0.41	 CRISIS < -0.6
--	--	--	--	--	---

⚠ **CRISIS flag:** any student scoring CRISIS must trigger an immediate alert and set `is_at_risk=True`.

Q2.1 — 6-Tier Wellbeing Classifier

Implement `assess_wellbeing(text)` using VADER. Return a dict with keys: `{'tier': str, 'score': float, 'emoji': str, 'is_at_risk': bool}`. Set `is_at_risk=True` only for CRISIS tier. [12 marks]

```
from nltk.sentiment import SentimentIntensityAnalyzer
```

```

sia = SentimentIntensityAnalyzer()

WELLBEING_SCALE = [
    ('THRIVING', 0.60, float('inf'), '🌟'),
    ('CONTENT', 0.20, 0.60, '😊'),
    ('NEUTRAL', -0.19, 0.20, '😐'),
    ('STRESSED', -0.40, -0.19, '😞'),
    ('DISTRESSED', -0.60, -0.40, '😓'),
    ('CRISIS', float('-inf'), -0.60, '🆘'),
]

def assess_wellbeing(text):
    # YOUR CODE HERE

    # Use sia.polarity_scores(text)['compound']

    # Match against WELLBEING_SCALE

    # Set is_at_risk=True if tier == 'CRISIS'

    pass

```

Q2.2 — Wellbeing Trajectory

Write `compute_trajectory(wellbeing_log)` where `wellbeing_log` is a list of dicts from Q2.1. Return: `{'trend': str, 'lowest_tier': str, 'at_risk_turns': list}`. `trend` is `'improving'`, `'declining'`, or `'fluctuating'` based on comparing first-half vs second-half average scores. **[8 marks]**

Trajectory Logic:

- Split log into first half and second half
- Compute mean score for each half
- If `second_avg > first_avg + 0.1` → `'improving'`
- If `second_avg < first_avg - 0.1` → `'declining'`
- Otherwise → `'fluctuating'`
- `at_risk_turns`: list of turn indices (0-based) where `is_at_risk == True`

Q2.3 — At-Risk Alert System

Write `check_and_alert(wellbeing_result, turn_number)`. If `is_at_risk` is True, print a prominent alert message recommending the student contact the university counselling line. The alert must include the turn number. Return the boolean flag. **[5 marks]**

Stage 3 — Support Classifier & At-Risk Flag System

20 minutes | 30 marks total

NEXUS must understand what type of support the student needs. Unlike a customer chatbot, student needs are more nuanced — one student might start asking about a deadline (academic) and shift to expressing mental health concerns (wellbeing). NEXUS tracks all of this.

NEXUS Support Categories

Category	Trigger Keywords	Typical Student Message
ACADEMIC	assignment, deadline, exam, grade, fail, pass, lecture, study, professor, submit	<i>I failed my midterm and don't know what to do</i>
WELLBEING	stress, anxious, depressed, lonely, overwhelmed, panic, cry, hopeless, afraid	<i>I can't sleep and I feel completely overwhelmed</i>
FINANCIAL	fees, scholarship, loan, afford, money, rent, bursary, payment, debt	<i>I can't afford my accommodation fees this semester</i>
TECHNICAL	portal, login, password, system, error, access, email, VPN, reset	<i>The student portal won't let me log in to submit</i>
SOCIAL	friends, roommate, belong, isolated, group, relationship, community	<i>I don't have any friends here and feel very alone</i>
ADMIN	enrolment, certificate, transcript, registration, form, office, HR	<i>I need my transcript urgently for a job application</i>

Q3.1 — Multi-Label Support Classifier

Implement `classify_support_need(text)`. Unlike Stage 3 of other chatbots, this function must return **ALL matching categories** (a student may need ACADEMIC + WELLBEING sup-

port simultaneously). Return a dict: {'primary': str, 'all_detected': list, 'scores': dict}. Primary is the highest-scoring category. **[12 marks]**

```
SUPPORT_KEYWORDS = {
    'ACADEMIC': ['assignment', 'deadline', 'exam', 'grade', 'fail', 'pass',
                 'lecture', 'study', 'professor', 'submit'],
    'WELLBEING': ['stress', 'anxious', 'depressed', 'lonely', 'overwhelmed',
                 'panic', 'cry', 'hopeless', 'afraid'],
    'FINANCIAL': ['fees', 'scholarship', 'loan', 'afford', 'money', 'rent',
                 'bursary', 'payment', 'debt'],
    'TECHNICAL': ['portal', 'login', 'password', 'system', 'error', 'access',
                 'email', 'VPN', 'reset'],
    'SOCIAL': ['friends', 'roommate', 'belong', 'isolated', 'group',
              'relationship', 'community'],
    'ADMIN': ['enrolment', 'certificate', 'transcript', 'registration',
             'form', 'office'],
}

def classify_support_need(text):
    # YOUR CODE HERE

    # Score ALL categories

    # Return primary + all_detected (score > 0) + scores dict
    pass
```

Q3.2 — Support Transition Logger

Write `log_support_transition(support_log, new_primary, turn_number)`. Append a transition event if the primary support category changed from the previous turn. Event dict: {'prev': str, 'curr': str, 'turn': int, 'is_escalation': bool}. Mark `is_escalation=True` if the transition goes into WELLBEING from any other category. **[10 marks]**

Q3.3 — Empathetic Response Generator

Write `nexus_respond(text, support_need, wellbeing)` where `support_need` is the primary category and `wellbeing` is the tier string. Handle at minimum 6 specific combinations below. **[8 marks]**

Required combinations — your function MUST produce distinct responses for each:

- WELLBEING + CRISIS → 'I am very concerned about you. Please contact the university counselling line RIGHT NOW: 0800-XXX-XXXX.'
- WELLBEING + DISTRESSED → 'It sounds like you are going through a really difficult time. Have you spoken to anyone about how you are feeling?'
- ACADEMIC + STRESSED → 'Exam pressure is real. Let us look at what support your faculty offers — have you spoken to your tutor?'
- FINANCIAL + any → 'Financial difficulty is more common than you think. The university bursary office can help — shall I give you their contact?'
- TECHNICAL + any → 'Let me help you with that technical issue. Which system are you trying to access?'
- SOCIAL + DISTRESSED → 'Feeling isolated at university is incredibly hard. The student union runs weekly social events — would that help?'

Stage 4 — SURPRISE CHALLENGE

15 minutes | 15 marks total

Q4 — Offline Session Replay (15 marks): Process the conversation log below through your complete NEXUS pipeline. No microphone needed — treat each line as text input. For every turn print: wellbeing emoji + tier, primary support category, trajectory update, and NEXUS response. Print the final intelligence report at the end.

```
STUDENT_LOG = [  
    'Hi, I need some help please.',  
    'I have a major assignment due tomorrow and I have not started.',  
    'My laptop also broke yesterday so I cannot access my files.',  
    'To be honest I have been struggling a lot lately, not just academically.',  
    'I have not been sleeping, I feel completely hopeless about everything.',  
    'I think I might need to talk to someone but I do not know who.',  
    'Also I got an email saying my fees are overdue and I cannot register.',  
    'Sorry for dumping all this. I just feel very alone right now.',  
    'Actually, my friend just texted. I feel a tiny bit better now.',  
    'Thank you for listening. I will try to contact the counsellor.',  
]
```

```
# For each message in STUDENT_LOG:
prev_tier = 'NEUTRAL'
for i, msg in enumerate(STUDENT_LOG):
    wb      = assess_wellbeing(msg)
    need    = classify_support_need(msg)
    alert   = check_and_alert(wb, i+1)
    resp    = nexus_respond(msg, need['primary'], wb['tier'])
    print(f'Turn {i+1}: {wb["emoji"]} {wb["tier"]} | {need["primary"]}
          | all: {need["all_detected"]}')
    print(f'NEXUS: {resp}\n')
    prev_tier = wb['tier']
```

Marks: 5 — correct wellbeing tiers for all 10 turns | 5 — correct primary + multi-label categories | 5 — appropriate NEXUS responses (especially turns 5, 6, 8)

Stage 5 — Intelligence Report & Submission

15 minutes | 10 marks total

NEXUS ends every session by generating a structured intelligence report for the university counselling team. This is NOT a summary for the student — it is a clinical handover document.

Q5.1 — Counsellor Intelligence Report

Implement `generate_intelligence_report(session_log, wellbeing_log, support_log, transition_log)`. Print the full report to console. **[10 marks]**

Required Report Fields:

- Session turns total (voice turns vs text turns listed separately)
- Wellbeing trajectory: IMPROVING / DECLINING / FLUCTUATING
- Lowest wellbeing tier reached during session
- Total at-risk alerts triggered (list which turns)
- All support categories detected, ranked by frequency
- Support escalation events (transitions into WELLBEING)
- Risk Score (0–100) — formula provided below
- Recommended action: NO_ACTION / FOLLOW_UP / URGENT_REFERRAL

Risk Score Formula :

```
base_risk          = 20

wellbeing_penalty = abs(avg_wellbeing_score) * 40 # worse score = higher risk

at_risk_penalty    = at_risk_count * 15           # +15 per CRISIS turn

escalation_penalty = escalation_count * 10         # +10 per WELLBEING escalation

word_count_factor = 0 # if avg words/turn > 20: bonus risk += 5 (verbose = distressed)

risk_score = base_risk + wellbeing_penalty + at_risk_penalty + escalation_penalty

risk_score = max(0, min(100, round(risk_score)))

# Action thresholds:

# risk_score >= 70 → 'URGENT_REFERRAL'

# 40 <= score < 70 → 'FOLLOW_UP'

# score < 40      → 'NO_ACTION'
```

```
def generate_intelligence_report(session_log, wellbeing_log,
                                support_log, transition_log):

    print('=' * 55)

    print('  NEXUS STUDENT INTELLIGENCE REPORT')

    print('  Code: NX-2B  -  Counsellor Eyes Only')

    print('=' * 55)

    # YOUR CODE HERE

    # Compute all required fields

    # Calculate risk score

    # Print recommended action

    print('=' * 55)
```

Submission Details:

- Deadline: Wednesday, May 20, 2026, upto 5:30PM.
- You must submit code files:
 - hci_assessment_YourName.py — your complete, working Python code
- Submit using shared Google Form Link in Classroom within the given time.